

AI For Math Seminar, ZJU

Lec 03: Lean AI — instance

Mulikas

Outline

Mathlib AI

- **instance** Lean

Mathlib AI

- **instance** Lean
- **Mathlib**

Mathlib AI

- **instance** Lean
- **Mathlib**
- **AI** LLM Lean

— instance Lean

```
G : Type*   [Group G]   mul_assoc
              Lean
```

— instance Lean

```
G : Type*   [Group G]   mul_assoc
              Lean
```



— instance Lean

```
G : Type*   [Group G]   mul_assoc
              Lean
```

-
- [...] Lean instance

— instance Lean

```
G : Type*   [Group G]   mul_assoc
              Lean
```

-
- [...] Lean instance
- typeclass

— instance Lean

G : Type*	[Group G]	mul_assoc
	Lean	

-
- [...] Lean instance
- typeclass

```
variable (G : Type*) [Group G]
```

```
#synth Group G
```

```
#check mul_assoc
```

```
#check inv_mul_cancel
```


- `mul, one, inv`

- mul, one, inv
- add, zero, neg

- mul, one, inv
- add, zero, neg
- Hom, id, comp

- mul, one, inv
- add, zero, neg
- Hom, id, comp

goal

- mul, one, inv
- add, zero, neg
- Hom, id, comp

•

goal

- mul, one, inv
- add, zero, neg
- Hom, id, comp

•

•

goal

- mul, one, inv
- add, zero, neg
- Hom, id, comp

•

•

•

goal

- mul, one, inv
- add, zero, neg
- Hom, id, comp

-
-
-

goal

```
#print      := by
```

Outline

AddGroup / Group — AddGroup Group

AddGroup A	0, +, -a	zero, add, neg
Group G	1, *, g^{-1}	one, mul, inv

```
#check AddGroup
```

```
#check Group
```

```
#check neg_add_cancel
```

```
#check inv_mul_cancel
```

```
example (A : Type*) [AddGroup A] (a : A) :  
  -a + a = 0 := neg_add_cancel a
```

```
example (G : Type*) [Group G] (g : G) :  
  g1 * g = 1 := inv_mul_cancel g
```

```
class / / Lean /
```

— AddGroup

— AddGroup

— AddGroup

- [Group G]

— AddGroup

- [Group G]
- $a * b$

— AddGroup

- [Group G]
- $a * b$
- `inv_mul_cancel`

— AddGroup

- [Group G]
- $a * b$
- `inv_mul_cancel`
-

— AddGroup

- [Group G]
- $a * b$
- inv_mul_cancel
-

— AddGroup

- [Group G]
- $a * b$
- `inv_mul_cancel`
-
- [AddCommGroup V]

— AddGroup

- [Group G]
 - $a * b$
 - `inv_mul_cancel`
 -
- [AddCommGroup V]
 - $u + v$

— AddGroup

- [Group G]
 - $a * b$
 - `inv_mul_cancel`
 -
- [AddCommGroup V]
 - $u + v$
 - `neg_add_cancel`

— AddGroup

- [Group G]
- $a * b$
- `inv_mul_cancel`
-
- [AddCommGroup V]
- $u + v$
- `neg_add_cancel`
- [Module $K\ V$]

— AddGroup

- [Group G]
- $a * b$
- inv_mul_cancel
-
- [AddCommGroup V]
- $u + v$
- neg_add_cancel
- [Module K V]

AddGroup

Monoid — Group

mul	$G \rightarrow G \rightarrow G$	
one		
inv	$G \rightarrow G$	
mul_assoc	cases /	/ group
one_mul, mul_one	rfl	
inv_mul_cancel		

- mul, one, inv —

mul	$G \rightarrow G \rightarrow G$	
one		
inv	$G \rightarrow G$	
mul_assoc	cases /	/ group
one_mul, mul_one	rfl	
inv_mul_cancel		

- mul, one, inv —
- mul_assoc, one_mul, mul_one, inv_mul_cancel —

Monoid — Group

<code>mul</code>	$G \rightarrow G \rightarrow G$	
<code>one</code>		
<code>inv</code>	$G \rightarrow G$	
<code>mul_assoc</code>	<code>cases /</code>	<code>/ group</code>
<code>one_mul, mul_one</code>	<code>rfl</code>	
<code>inv_mul_cancel</code>		

- `mul, one, inv` —
- `mul_assoc, one_mul, mul_one, inv_mul_cancel` —
- `Lean` instance

Sign —

```
inductive Sign where
  | pos
  | neg
  deriving DecidableEq, Repr

open Sign

instance : Group Sign where
  mul a b :=
    match a, b with
    | pos, x => x
    | neg, pos => neg
    | neg, neg => pos
  one := pos
  inv a := a
  -- laws come next
```


Sign —

```
inductive Sign where
  | pos
  | neg
  deriving DecidableEq, Repr
```

```
open Sign
```

```
instance : Group Sign where
  mul a b :=
    match a, b with
    | pos, x => x
    | neg, pos => neg
    | neg, neg => pos
  one := pos
  inv a := a
  -- laws come next
```

Sign —

• pos

```
inductive Sign where
  | pos
  | neg
  deriving DecidableEq, Repr
```

```
open Sign
```

```
instance : Group Sign where
  mul a b :=
    match a, b with
    | pos, x => x
    | neg, pos => neg
    | neg, neg => pos
  one := pos
  inv a := a
  -- laws come next
```

Sign —

```
inductive Sign where
  | pos
  | neg
  deriving DecidableEq, Repr
```

```
open Sign
```

```
instance : Group Sign where
  mul a b :=
    match a, b with
    | pos, x => x
    | neg, pos => neg
    | neg, neg => pos
  one := pos
  inv a := a
  -- laws come next
```

- pos
- $\text{neg} * \text{neg} = \text{pos}$

Sign —

```
inductive Sign where
  | pos
  | neg
  deriving DecidableEq, Repr
```

```
open Sign
```

```
instance : Group Sign where
  mul a b :=
    match a, b with
    | pos, x => x
    | neg, pos => neg
    | neg, neg => pos
  one := pos
  inv a := a
  -- laws come next
```

- pos
- $\text{neg} * \text{neg} = \text{pos}$
-

Sign —

```
inductive Sign where
  | pos
  | neg
  deriving DecidableEq, Repr
```

```
open Sign
```

```
instance : Group Sign where
  mul a b :=
    match a, b with
    | pos, x => x
    | neg, pos => neg
    | neg, neg => pos
  one := pos
  inv a := a
  -- laws come next
```

- pos
- $\text{neg} * \text{neg} = \text{pos}$
-
-

Sign —

```
inductive Sign where
  | pos
  | neg
  deriving DecidableEq, Repr
```

```
open Sign
```

```
instance : Group Sign where
  mul a b :=
    match a, b with
    | pos, x => x
    | neg, pos => neg
    | neg, neg => pos
  one := pos
  inv a := a
  -- laws come next
```

- pos
- $\text{neg} * \text{neg} = \text{pos}$
-
-

```
pos * neg
neg * neg
```



```
mul_assoc := by
  intro a b c
  cases a <;> cases b
  <;> cases c <;> rfl
one_mul := by
  intro a
  cases a <;> rfl
mul_one := by
  intro a
  cases a <;> rfl
inv_mul_cancel := by
  intro a
  cases a <;> rfl
```

#synth Group Sign

```
example (a : Sign) :
  a1 * a = 1 := by
  exact inv_mul_cancel a
```



```
mul_assoc := by
  intro a b c
  cases a <;> cases b
    <;> cases c <;> rfl
one_mul := by
  intro a
  cases a <;> rfl
mul_one := by
  intro a
  cases a <;> rfl
inv_mul_cancel := by
  intro a
  cases a <;> rfl
```

#synth Group Sign

```
example (a : Sign) :
  a1 * a = 1 := by
  exact inv_mul_cancel a
```

① intro

```
mul_assoc := by
  intro a b c
  cases a <;> cases b
  <;> cases c <;> rfl
one_mul := by
  intro a
  cases a <;> rfl
mul_one := by
  intro a
  cases a <;> rfl
inv_mul_cancel := by
  intro a
  cases a <;> rfl
```

#synth Group Sign

```
example (a : Sign) :
  a1 * a = 1 := by
  exact inv_mul_cancel a
```

```

mul_assoc := by
  intro a b c
  cases a <;> cases b
    <;> cases c <;> rfl
one_mul := by
  intro a
  cases a <;> rfl
mul_one := by
  intro a
  cases a <;> rfl
inv_mul_cancel := by
  intro a
  cases a <;> rfl

```

#synth Group Sign

```

example (a : Sign) :
  a1 * a = 1 := by
  exact inv_mul_cancel a

```

① intro

② cases pos neg

```
mul_assoc := by
  intro a b c
  cases a <;> cases b
    <;> cases c <;> rfl
one_mul := by
  intro a
  cases a <;> rfl
mul_one := by
  intro a
  cases a <;> rfl
inv_mul_cancel := by
  intro a
  cases a <;> rfl
```

#synth Group Sign

```
example (a : Sign) :
  a1 * a = 1 := by
  exact inv_mul_cancel a
```

- ① intro
- ② cases pos neg
- ③ rfl

Group — cases

```
mul_assoc := by
  intro a b c
  cases a <;> cases b
    <;> cases c <;> rfl
one_mul := by
  intro a
  cases a <;> rfl
mul_one := by
  intro a
  cases a <;> rfl
inv_mul_cancel := by
  intro a
  cases a <;> rfl
```

#synth Group Sign

```
example (a : Sign) :
  a1 * a = 1 := by
  exact inv_mul_cancel a
```

- ① intro
- ② cases pos neg
- ③ rfl
- ④ Group

Group — cases

```
mul_assoc := by
  intro a b c
  cases a <;> cases b
    <;> cases c <;> rfl
one_mul := by
  intro a
  cases a <;> rfl
mul_one := by
  intro a
  cases a <;> rfl
inv_mul_cancel := by
  intro a
  cases a <;> rfl
```

#synth Group Sign

```
example (a : Sign) :
  a1 * a = 1 := by
  exact inv_mul_cancel a
```

- ① intro
- ② cases pos neg
- ③ rfl
- ④ Group

+ cases + rfl

=

Outline

— Subtype

— Subtype

```
def NonzeroInt :=  
  {n : // n 0}  
  
example : NonzeroInt :=  
  1, by norm_num  
  
example (x : NonzeroInt) : :=  
  x.1  
  
example (x : NonzeroInt) :  
  x.1 0 := x.property  
  
#check Subtype.val  
#check Subtype.property
```

— Subtype

```
def NonzeroInt :=  
  {n : // n 0}  
  
example : NonzeroInt :=  
  1, by norm_num  
  
example (x : NonzeroInt) : :=  
  x.1  
  
example (x : NonzeroInt) :  
  x.1 0 := x.property  
  
#check Subtype.val  
#check Subtype.property
```

— Subtype

```
def NonzeroInt :=  
  {n : // n 0}
```

• x.1

```
example : NonzeroInt :=  
  1, by norm_num
```

```
example (x : NonzeroInt) : :=  
  x.1
```

```
example (x : NonzeroInt) :  
  x.1 0 := x.property
```

```
#check Subtype.val  
#check Subtype.property
```

— Subtype

```
def NonzeroInt :=  
  {n : // n 0}
```

- x.1

```
example : NonzeroInt :=  
  1, by norm_num
```

- x.2 x.property

```
example (x : NonzeroInt) : :=  
  x.1
```

```
example (x : NonzeroInt) :  
  x.1 0 := x.property
```

```
#check Subtype.val  
#check Subtype.property
```

— Subtype

```
def NonzeroInt :=  
  {n : // n 0}
```

```
example : NonzeroInt :=  
  1, by norm_num
```

```
example (x : NonzeroInt) : :=  
  x.1
```

```
example (x : NonzeroInt) :  
  x.1 0 := x.property
```

```
#check Subtype.val  
#check Subtype.property
```

- x.1
- x.2 x.property
- value, proof

— Subtype

```
def NonzeroInt :=  
  {n : // n 0}  
  
example : NonzeroInt :=  
  1, by norm_num  
  
example (x : NonzeroInt) : :=  
  x.1  
  
example (x : NonzeroInt) :  
  x.1 0 := x.property  
  
#check Subtype.val  
#check Subtype.property
```

- x.1
- x.2 x.property
- value, proof
-

— Subtype

```
def NonzeroInt :=  
  {n : // n 0}
```

```
example : NonzeroInt :=  
  1, by norm_num
```

```
example (x : NonzeroInt) : :=  
  x.1
```

```
example (x : NonzeroInt) :  
  x.1 0 := x.property
```

```
#check Subtype.val  
#check Subtype.property
```

- x.1
- x.2 x.property
- value, proof
-

Subtype " "

Group — Subgroup = carrier +

Group — Subgroup = carrier +

```
def oneSubgroup (G : Type*)
  [Group G] : Subgroup G
  where
    carrier := {g | g = 1}
    one_mem' := by rfl
    mul_mem' := by
      intro a b ha hb
      rw [ha, hb]; simp
    inv_mem' := by
      intro a ha
      rw [ha]; simp

example (G : Type*) [Group G]
  (x : oneSubgroup G) :
  (x : G) = 1 := x.property
```

Group — Subgroup = carrier +

```
def oneSubgroup (G : Type*)
  [Group G] : Subgroup G
  where
    carrier := {g | g = 1}
    one_mem' := by rfl
    mul_mem' := by
      intro a b ha hb
      rw [ha, hb]; simp
    inv_mem' := by
      intro a ha
      rw [ha]; simp

example (G : Type*) [Group G]
  (x : oneSubgroup G) :
  (x : G) = 1 := x.property
```

Group — Subgroup = carrier +

```
def oneSubgroup (G : Type*)  
  [Group G] : Subgroup G  
  where  
    carrier := {g | g = 1}  
    one_mem' := by rfl  
    mul_mem' := by  
      intro a b ha hb  
      rw [ha, hb]; simp  
    inv_mem' := by  
      intro a ha  
      rw [ha]; simp
```

① carrier g : G

```
example (G : Type*) [Group G]  
  (x : oneSubgroup G) :  
  (x : G) = 1 := x.property
```

Group — Subgroup = carrier +

```
def oneSubgroup (G : Type*)  
  [Group G] : Subgroup G  
  where  
    carrier := {g | g = 1}  
    one_mem' := by rfl  
    mul_mem' := by  
      intro a b ha hb  
      rw [ha, hb]; simp  
    inv_mem' := by  
      intro a ha  
      rw [ha]; simp  
  
example (G : Type*) [Group G]  
  (x : oneSubgroup G) :  
  (x : G) = 1 := x.property
```

- ① carrier g : G
- ② one_mem'

Group — Subgroup = carrier +

```
def oneSubgroup (G : Type*)  
  [Group G] : Subgroup G  
  where  
    carrier := {g | g = 1}  
    one_mem' := by rfl  
    mul_mem' := by  
      intro a b ha hb  
      rw [ha, hb]; simp  
    inv_mem' := by  
      intro a ha  
      rw [ha]; simp
```

```
example (G : Type*) [Group G]  
  (x : oneSubgroup G) :  
  (x : G) = 1 := x.property
```

- ① carrier g : G
- ② one_mem'
- ③ mul_mem'

Group — Subgroup = carrier +

```
def oneSubgroup (G : Type*)  
  [Group G] : Subgroup G  
  where  
    carrier := {g | g = 1}  
    one_mem' := by rfl  
    mul_mem' := by  
      intro a b ha hb  
      rw [ha, hb]; simp  
    inv_mem' := by  
      intro a ha  
      rw [ha]; simp  
  
example (G : Type*) [Group G]  
  (x : oneSubgroup G) :  
  (x : G) = 1 := x.property
```

- ① carrier g : G
- ② one_mem'
- ③ mul_mem'
- ④ inv_mem'

Group — Subgroup = carrier +

```
def oneSubgroup (G : Type*)  
  [Group G] : Subgroup G  
  where  
    carrier := {g | g = 1}  
    one_mem' := by rfl  
    mul_mem' := by  
      intro a b ha hb  
      rw [ha, hb]; simp  
    inv_mem' := by  
      intro a ha  
      rw [ha]; simp  
  
example (G : Type*) [Group G]  
  (x : oneSubgroup G) :  
  (x : G) = 1 := x.property
```

- ① carrier g : G
- ② one_mem'
- ③ mul_mem'
- ④ inv_mem'
- ⑤ Mathlib

Group — Subgroup = carrier +

```
def oneSubgroup (G : Type*)
  [Group G] : Subgroup G
  where
    carrier := {g | g = 1}
    one_mem' := by rfl
    mul_mem' := by
      intro a b ha hb
      rw [ha, hb]; simp
    inv_mem' := by
      intro a ha
      rw [ha]; simp

example (G : Type*) [Group G]
  (x : oneSubgroup G) :
  (x : G) = 1 := x.property
```

- ① carrier g : G
- ② one_mem'
- ③ mul_mem'
- ④ inv_mem'
- ⑤ Mathlib

Subgroup
Lean

Outline

Lean

0, +, -, / /	AddCommGroup
1, *, /	Monoid
0 0	left_distrib, right_distrib
	CommRing / mul_comm

Ring

PairInt — Ring instance

PairInt — Ring instance

```
@[ext]
structure PairInt where
  x : Int
  y : Int

def equivProd : PairInt → Int × Int
  where
    toFun p      := (p.x, p.y)
    invFun q     := q.1, q.2
    left_inv     := by intro p; ext
                  <;> rfl
    right_inv    := by intro q; cases
                  q; rfl

instance : CommRing PairInt :=
  equivProd.commRing
```

PairInt — Ring instance

```
@[ext]
structure PairInt where
  x : Int
  y : Int

def equivProd : PairInt → Int × Int
  where
    toFun p      := (p.x, p.y)
    invFun q     := q.1, q.2
    left_inv     := by intro p; ext
                  <;> rfl
    right_inv    := by intro q; cases
                  q; rfl

instance : CommRing PairInt :=
  equivProd.commRing
```

PairInt — Ring instance

```
@[ext]
structure PairInt where
  x : Int
  y : Int

def equivProd : PairInt → Int × Int
  where
    toFun p      := (p.x, p.y)
    invFun q     := q.1, q.2
    left_inv     := by intro p; ext
                  <;> rfl
    right_inv    := by intro q; cases
                  q; rfl

instance : CommRing PairInt :=
  equivProd.commRing
```

• $\text{Int} \times \text{Int}$

PairInt — Ring instance

```
@[ext]
structure PairInt where
  x : Int
  y : Int

def equivProd : PairInt → Int × Int
  where
    toFun p      := (p.x, p.y)
    invFun q     := q.1, q.2
    left_inv     := by intro p; ext
                  <;> rfl
    right_inv    := by intro q; cases
                  q; rfl

instance : CommRing PairInt :=
  equivProd.commRing
```

- $\text{Int} \times \text{Int}$
- PairInt

PairInt — Ring instance

```
@[ext]
structure PairInt where
  x : Int
  y : Int

def equivProd : PairInt → Int × Int
  where
    toFun p      := (p.x, p.y)
    invFun q     := q.1, q.2
    left_inv     := by intro p; ext
                  <;> rfl
    right_inv    := by intro q; cases
                  q; rfl

instance : CommRing PairInt :=
  equivProd.commRing
```

- $\text{Int} \times \text{Int}$
- PairInt
- Equiv.commRing

PairInt — Ring instance

```
@[ext]
structure PairInt where
  x : Int
  y : Int

def equivProd : PairInt → Int × Int
  where
    toFun p      := (p.x, p.y)
    invFun q     := q.1, q.2
    left_inv     := by intro p; ext
                  <;> rfl
    right_inv    := by intro q; cases
                  q; rfl

instance : CommRing PairInt :=
  equivProd.commRing
```

- $\text{Int} \times \text{Int}$
- PairInt
- Equiv.commRing
-


```
#synth CommRing PairInt
#check add_assoc
#check neg_add_cancel
#check left_distrib
#check right_distrib
#check mul_assoc
#check mul_comm
example (p q r : PairInt) :
  p * (q + r) = p * q + p * r :=
  by
    exact mul_add p q r
example (p q : PairInt) :
  (p + q)^2 = p^2 + 2*p*q + q^2
  := by
  ring
```

```
#synth CommRing PairInt
#check add_assoc
#check neg_add_cancel
#check left_distrib
#check right_distrib
#check mul_assoc
#check mul_comm
example (p q r : PairInt) :
  p * (q + r) = p * q + p * r :=
  by
    exact mul_add p q r
example (p q : PairInt) :
  (p + q)^2 = p^2 + 2*p*q + q^2
  := by
  ring
```

```
#synth CommRing PairInt
#check add_assoc
#check neg_add_cancel
#check left_distrib
#check right_distrib
#check mul_assoc
#check mul_comm
example (p q r : PairInt) :
  p * (q + r) = p * q + p * r :=
  by
    exact mul_add p q r
example (p q : PairInt) :
  (p + q)^2 = p^2 + 2*p*q + q^2
  := by
  ring
```

① #synth

```
#synth CommRing PairInt
#check add_assoc
#check neg_add_cancel
#check left_distrib
#check right_distrib
#check mul_assoc
#check mul_comm
example (p q r : PairInt) :
  p * (q + r) = p * q + p * r :=
  by
    exact mul_add p q r
example (p q : PairInt) :
  (p + q)^2 = p^2 + 2*p*q + q^2
  := by
  ring
```

① #synth

② #check

```
#synth CommRing PairInt
#check add_assoc
#check neg_add_cancel
#check left_distrib
#check right_distrib
#check mul_assoc
#check mul_comm
example (p q r : PairInt) :
  p * (q + r) = p * q + p * r :=
  by
    exact mul_add p q r
example (p q : PairInt) :
  (p + q)^2 = p^2 + 2*p*q + q^2
  := by
  ring
```

① #synth

② #check

③

```
#synth CommRing PairInt
#check add_assoc
#check neg_add_cancel
#check left_distrib
#check right_distrib
#check mul_assoc
#check mul_comm
example (p q r : PairInt) :
  p * (q + r) = p * q + p * r :=
  by
    exact mul_add p q r
example (p q : PairInt) :
  (p + q)^2 = p^2 + 2*p*q + q^2
  := by
  ring
```

- ① #synth
- ② #check
- ③
- ④ ring


```
def diagonalSubring : Subring ( ×
) where
  carrier := {p | p.1 = p.2}
  zero_mem' := by simp
  one_mem' := by simp
  add_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  neg_mem' := by
    intro a ha; simp at ha ; rw
    [ha]
  mul_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
example (x : diagonalSubring) :
  (x : × ).1 = (x : × ).2 :=
  x.property
```

```
def diagonalSubring : Subring ( ×
) where
  carrier := {p | p.1 = p.2}
  zero_mem' := by simp
  one_mem' := by simp
  add_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  neg_mem' := by
    intro a ha; simp at ha ; rw
    [ha]
  mul_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
example (x : diagonalSubring) :
  (x : × ).1 = (x : × ).2 :=
  x.property
```

```
def diagonalSubring : Subring ( ×
  ) where
  carrier := {p | p.1 = p.2}
  zero_mem' := by simp
  one_mem' := by simp
  add_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  neg_mem' := by
    intro a ha; simp at ha ; rw
    [ha]
  mul_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
example (x : diagonalSubring) :
  (x : × ).1 = (x : × ).2 :=
  x.property
```

• $p.1 = p.2$ pair

```
def diagonalSubring : Subring ( ×
) where
  carrier := {p | p.1 = p.2}
  zero_mem' := by simp
  one_mem' := by simp
  add_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  neg_mem' := by
    intro a ha; simp at ha ; rw
    [ha]
  mul_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
example (x : diagonalSubring) :
  (x : × ).1 = (x : × ).2 :=
  x.property
```

- $p.1 = p.2$ pair
- zero_mem'
- one_mem'

```
def diagonalSubring : Subring ( ×
) where
  carrier := {p | p.1 = p.2}
  zero_mem' := by simp
  one_mem' := by simp
  add_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  neg_mem' := by
    intro a ha; simp at ha ; rw
    [ha]
  mul_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
example (x : diagonalSubring) :
  (x : × ).1 = (x : × ).2 :=
  x.property
```

- $p.1 = p.2$ pair
- zero_mem'
- one_mem'
- add_mem', neg_mem', mul_mem'

```
def diagonalSubring : Subring ( ×
) where
  carrier := {p | p.1 = p.2}
  zero_mem' := by simp
  one_mem' := by simp
  add_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  neg_mem' := by
    intro a ha; simp at ha ; rw
    [ha]
  mul_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
example (x : diagonalSubring) :
  (x : × ).1 = (x : × ).2 :=
  x.property
```

- $p.1 = p.2$ pair
- zero_mem'
- one_mem'
- add_mem', neg_mem', mul_mem'
- x.property

Ring — Subring Subgroup

```
def diagonalSubring : Subring ( ×
) where
  carrier := {p | p.1 = p.2}
  zero_mem' := by simp
  one_mem' := by simp
  add_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  neg_mem' := by
    intro a ha; simp at ha ; rw
    [ha]
  mul_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
example (x : diagonalSubring) :
  (x : × ).1 = (x : × ).2 :=
  x.property
```

- $p.1 = p.2$ pair
- zero_mem'
- one_mem'
- add_mem', neg_mem', mul_mem'
- x.property
-

Outline

— Field = +

	Lean
CommRing K	ring
Nontrivial K	$0 \neq 1$
inv, div	$x / y = x * y^{-1}$
mul_inv_cancel	$x \neq 0$
inv_zero	Lean $0^{-1} = 0$
AI	$x / x = 1$ $x \neq 0$


```

@[ext]
structure WrappedQ where
  val : Rat

def equivRat : Equiv WrappedQ Rat
  where
    toFun x    := x.val
    invFun q    := { val := q }
    left_inv    := by intro x; ext; rfl
    right_inv   := by intro q; rfl

instance : Field WrappedQ :=
  equivRat.field

```

```

@[ext]
structure WrappedQ where
  val : Rat

def equivRat : Equiv WrappedQ Rat
  where
    toFun x      := x.val
    invFun q     := { val := q }
    left_inv     := by intro x; ext; rfl
    right_inv    := by intro q; rfl

instance : Field WrappedQ :=
  equivRat.field
    
```

```

@[ext]
structure WrappedQ where
  val : Rat

def equivRat : Equiv WrappedQ Rat
  where
    toFun x      := x.val
    invFun q     := { val := q }
    left_inv     := by intro x; ext; rfl
    right_inv    := by intro q; rfl

instance : Field WrappedQ :=
  equivRat.field
    
```

• Rat

```

@[ext]
structure WrappedQ where
  val : Rat

def equivRat : Equiv WrappedQ Rat
  where
    toFun x    := x.val
    invFun q    := { val := q }
    left_inv    := by intro x; ext; rfl
    right_inv   := by intro q; rfl

instance : Field WrappedQ :=
  equivRat.field

```

- Rat
-

```

@[ext]
structure WrappedQ where
  val : Rat

def equivRat : Equiv WrappedQ Rat
  where
    toFun x      := x.val
    invFun q     := { val := q }
    left_inv     := by intro x; ext; rfl
    right_inv    := by intro q; rfl

instance : Field WrappedQ :=
  equivRat.field
    
```

- Rat
-
- field_simp [hx]


```

@[ext]
structure WrappedQ where
  val : Rat

def equivRat : Equiv WrappedQ Rat
  where
    toFun x      := x.val
    invFun q     := { val := q }
    left_inv     := by intro x; ext; rfl
    right_inv    := by intro q; rfl

instance : Field WrappedQ :=
  equivRat.field
    
```

- Rat
-
- field_simp [hx]
- statement

Outline

Lean

```
[Semiring R]  [Field K]
[AddCommGroup V]
r • v        [Module R V]
smul_add, add_smul, mul_smul, one_smul
```

```
variable {K : Type*} [Field K]
variable {V : Type*} [AddCommGroup V] [Module K V]
#check smul_add      -- r • (v + w) = r • v + r • w
#check add_smul      -- (r + s) • v = r • v + s • v
#check mul_smul      -- (r * s) • v = r • (s • v)
#check one_smul      -- (1 : K) • v = v
```



```

def diagonalSubmodule :
  Submodule ( × ) where
  carrier := {p | p.1 = p.2}
  zero_mem' := by simp
  add_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  smul_mem' := by
    intro c p hp
    change (c * p.1 = c * p.2)
    rw [hp]
example (x : diagonalSubmodule) :
  (x : × ).1 = (x : × ).2 :=
  x.property

```

```

def diagonalSubmodule :
  Submodule ( × ) where
  carrier      := {p | p.1 = p.2}
  zero_mem'    := by simp
  add_mem'     := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  smul_mem'    := by
    intro c p hp
    change (c * p.1 = c * p.2)
    rw [hp]
example (x : diagonalSubmodule) :
  (x : × ).1 = (x : × ).2 :=
  x.property

```

```

def diagonalSubmodule :
  Submodule ( × ) where
    carrier      := {p | p.1 = p.2}
    zero_mem'    := by simp
    add_mem'     := by
      intro a b ha hb
      simp at ha hb ; rw [ha, hb]
    smul_mem'    := by
      intro c p hp
      change (c * p.1 = c * p.2)
      rw [hp]
example (x : diagonalSubmodule) :
  (x : × ).1 = (x : × ).2 :=
  x.property

```

1 carrier

```

def diagonalSubmodule :
  Submodule ( × ) where
  carrier      := {p | p.1 = p.2}
  zero_mem'    := by simp
  add_mem'     := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  smul_mem'    := by
    intro c p hp
    change (c * p.1 = c * p.2)
    rw [hp]
example (x : diagonalSubmodule) :
  (x : × ).1 = (x : × ).2 :=
  x.property

```

- ① carrier
- ② zero_mem'


```

def diagonalSubmodule :
  Submodule ( × ) where
  carrier      := {p | p.1 = p.2}
  zero_mem'    := by simp
  add_mem'     := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  smul_mem'    := by
    intro c p hp
    change (c * p.1 = c * p.2)
    rw [hp]
example (x : diagonalSubmodule) :
  (x : × ).1 = (x : × ).2 :=
  x.property

```

- ① carrier
- ② zero_mem'
- ③ add_mem'

```

def diagonalSubmodule :
  Submodule ( × ) where
  carrier      := {p | p.1 = p.2}
  zero_mem'    := by simp
  add_mem'     := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  smul_mem'    := by
    intro c p hp
    change (c * p.1 = c * p.2)
    rw [hp]
example (x : diagonalSubmodule) :
  (x : × ).1 = (x : × ).2 :=
  x.property

```

- ① carrier
- ② zero_mem'
- ③ add_mem'
- ④ smul_mem'

```

def diagonalSubmodule :
  Submodule ( × ) where
  carrier      := {p | p.1 = p.2}
  zero_mem'    := by simp
  add_mem'     := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  smul_mem'    := by
    intro c p hp
    change (c * p.1 = c * p.2)
    rw [hp]
example (x : diagonalSubmodule) :
  (x : × ).1 = (x : × ).2 :=
  x.property

```

- ① carrier
- ② zero_mem'
- ③ add_mem'
- ④ smul_mem'
- ⑤

Outline

Symplectic vector space —

Symplectic vector space —

```
variable (F W : Type*)
variable [Field F] [Fintype F]
variable [AddCommGroup W] [Module
  F W]
variable [FiniteDimensional F W]

def IsAlternating
  ( : W → [F] W → [F] F) : Prop :=
  w : W,    w w = 0

def IsNondegenerate
  ( : W → [F] W → [F] F) : Prop :=
  w : W, ( v : W,    w v = 0) → w =
    0
```

Symplectic vector space —

```
variable (F W : Type*)
variable [Field F] [Fintype F]
variable [AddCommGroup W] [Module
  F W]
variable [FiniteDimensional F W]

def IsAlternating
  ( : W → [F] W → [F] F) : Prop :=
  w : W,    w w = 0

def IsNondegenerate
  ( : W → [F] W → [F] F) : Prop :=
  w : W, ( v : W,    w v = 0) → w =
    0
```

AI

Lean

Symplectic vector space —

```
variable (F W : Type*)
variable [Field F] [Fintype F]
variable [AddCommGroup W] [Module
  F W]
variable [FiniteDimensional F W]

def IsAlternating
  ( : W → [F] W → [F] F) : Prop :=
  w : W,    w w = 0

def IsNondegenerate
  ( : W → [F] W → [F] F) : Prop :=
  w : W, ( v : W,    w v = 0) → w =
    0
```

• W

AI

Lean

Symplectic vector space —

```
variable (F W : Type*)
variable [Field F] [Fintype F]
variable [AddCommGroup W] [Module
  F W]
variable [FiniteDimensional F W]

def IsAlternating
  ( : W → [F] W → [F] F) : Prop :=
  w : W,    w w = 0

def IsNondegenerate
  ( : W → [F] W → [F] F) : Prop :=
  w : W, ( v : W,    w v = 0) → w =
    0
```

• W

•

AI

Lean

Symplectic vector space —

```
variable (F W : Type*)
variable [Field F] [Fintype F]
variable [AddCommGroup W] [Module
  F W]
variable [FiniteDimensional F W]

def IsAlternating
  ( : W → [F] W → [F] F) : Prop :=
  w : W,   w w = 0

def IsNondegenerate
  ( : W → [F] W → [F] F) : Prop :=
  w : W, ( v : W,   w v = 0) → w =
    0
```

- W
-
- Prop

AI

Lean

Symplectic vector space —

```
variable (F W : Type*)
variable [Field F] [Fintype F]
variable [AddCommGroup W] [Module
  F W]
variable [FiniteDimensional F W]

def IsAlternating
  ( : W → [F] W → [F] F) : Prop :=
  w : W,   w w = 0

def IsNondegenerate
  ( : W → [F] W → [F] F) : Prop :=
  w : W, ( v : W,   w v = 0) → w =
    0
```

- W
-
- Prop
-

AI

Lean

Complete Polarization

CompletePolarization

```
def TotallyIsotropic
  ( : W → [F] W → [F] F)
  (U : Submodule F W) : Prop :=
  u U, v U, u v = 0

structure CompletePolarization
  ( : W → [F] W → [F] F) where
  X      : Submodule F W
  Y      : Submodule F W
  isotropic_X : TotallyIsotropic F
    W X
  isotropic_Y : TotallyIsotropic F
    W Y
  disjoint    : Disjoint X Y
  sup_eq_top  : X Y =
```

CompletePolarization

```
def TotallyIsotropic
  ( : W → [F] W → [F] F)
  (U : Submodule F W) : Prop :=
  u U, v U, u v = 0
```

```
structure CompletePolarization
  ( : W → [F] W → [F] F) where
  X      : Submodule F W
  Y      : Submodule F W
  isotropic_X : TotallyIsotropic F
    W X
  isotropic_Y : TotallyIsotropic F
    W Y
  disjoint    : Disjoint X Y
  sup_eq_top  : X Y =
```

sup_eq_top w : W

CompletePolarization

```
def TotallyIsotropic
  ( : W → [F] W → [F] F)
  (U : Submodule F W) : Prop :=
  u U, v U, u v = 0

structure CompletePolarization
  ( : W → [F] W → [F] F) where
  X      : Submodule F W
  Y      : Submodule F W
  isotropic_X : TotallyIsotropic F
    W X
  isotropic_Y : TotallyIsotropic F
    W Y
  disjoint    : Disjoint X Y
  sup_eq_top  : X Y =
```

• X, Y

sup_eq_top w : W

CompletePolarization

```
def TotallyIsotropic
  ( : W → [F] W → [F] F)
  (U : Submodule F W) : Prop :=
  u U, v U, u v = 0
```

```
structure CompletePolarization
  ( : W → [F] W → [F] F) where
  X      : Submodule F W
  Y      : Submodule F W
  isotropic_X : TotallyIsotropic F
    W X
  isotropic_Y : TotallyIsotropic F
    W Y
  disjoint    : Disjoint X Y
  sup_eq_top  : X Y =
```

- X, Y

-

sup_eq_top w : W

CompletePolarization

```
def TotallyIsotropic
  ( : W → [F] W → [F] F)
  (U : Submodule F W) : Prop :=
  u U, v U, u v = 0

structure CompletePolarization
  ( : W → [F] W → [F] F) where
  X      : Submodule F W
  Y      : Submodule F W
  isotropic_X : TotallyIsotropic F
    W X
  isotropic_Y : TotallyIsotropic F
    W Y
  disjoint    : Disjoint X Y
  sup_eq_top  : X Y =
```

- X, Y
-
- Disjoint X Y

sup_eq_top w : W

CompletePolarization

```
def TotallyIsotropic
  ( : W → [F] W → [F] F)
  (U : Submodule F W) : Prop :=
  u U, v U, u v = 0

structure CompletePolarization
  ( : W → [F] W → [F] F) where
  X      : Submodule F W
  Y      : Submodule F W
  isotropic_X : TotallyIsotropic F
    W X
  isotropic_Y : TotallyIsotropic F
    W Y
  disjoint    : Disjoint X Y
  sup_eq_top  : X Y =
```

- X, Y
-
- $\text{Disjoint } X \ Y$
- $X \ Y =$

$\text{sup_eq_top} \quad w : W$

Classical.choose —

```
#check Classical.choose
-- (h : x, p x) →
#check Classical.choose_spec
-- p (Classical.choose h)

lemma exists_decomposition
  (P : CompletePolarization F W )
  (w : W) :
  x P.X, y P.Y, w = x + y :=
by sorry

noncomputable def xComponent
  (P : CompletePolarization F W )
  (w : W) : W :=
Classical.choose
  (exists_decomposition F W P w)
```

Classical.choose —

```
#check Classical.choose
-- (h : x, p x) →
#check Classical.choose_spec
-- p (Classical.choose h)

lemma exists_decomposition
  (P : CompletePolarization F W )
  (w : W) :
    x P.X, y P.Y, w = x + y := " "
by sorry

noncomputable def xComponent
  (P : CompletePolarization F W )
  (w : W) : W :=
Classical.choose
  (exists_decomposition F W P w)
```

```
#check Classical.choose
-- (h : x, p x) →
#check Classical.choose_spec
-- p (Classical.choose h)

lemma exists_decomposition
  (P : CompletePolarization F W )
  (w : W) :
    x P.X, y P.Y, w = x + y :=
  by sorry

noncomputable def xComponent
  (P : CompletePolarization F W )
  (w : W) : W :=
  Classical.choose
    (exists_decomposition F W P w)
```

• $x, p\ x$

" "

```
#check Classical.choose
-- (h : x, p x) →
#check Classical.choose_spec
-- p (Classical.choose h)

lemma exists_decomposition
  (P : CompletePolarization F W )
  (w : W) :
    x P.X, y P.Y, w = x + y :=
  by sorry

noncomputable def xComponent
  (P : CompletePolarization F W )
  (w : W) : W :=
  Classical.choose
    (exists_decomposition F W P w)
```

- $x, p\ x$
- choose witness

" "

```
#check Classical.choose
-- (h : x, p x) →
#check Classical.choose_spec
-- p (Classical.choose h)

lemma exists_decomposition
  (P : CompletePolarization F W )
  (w : W) :
    x P.X, y P.Y, w = x + y :=
  by sorry

noncomputable def xComponent
  (P : CompletePolarization F W )
  (w : W) : W :=
  Classical.choose
    (exists_decomposition F W P w)
```

- $x, p\ x$
- choose witness
- choose_spec

" "


```
#check Classical.choose
-- (h : x, p x) →
#check Classical.choose_spec
-- p (Classical.choose h)

lemma exists_decomposition
  (P : CompletePolarization F W )
  (w : W) :
    x P.X, y P.Y, w = x + y :=
  by sorry

noncomputable def xComponent
  (P : CompletePolarization F W )
  (w : W) : W :=
  Classical.choose
    (exists_decomposition F W P w)
```

- $x, p\ x$
 - choose witness
 - choose_spec
 - noncomputable
- " "

choose_spec

choose_spec

```
noncomputable def yComponent
  (P : CompletePolarization F W )
  (w : W) : W :=
  Classical.choose
  (Classical.choose_spec
    (exists_decomposition F W P
      w)).2
```

```
lemma decompose_by_components
  (P : CompletePolarization F W )
  (w : W) :
  w = xComponent F W P w
    + yComponent F W P w := by
  dsimp [xComponent, yComponent]
  exact
  (Classical.choose_spec
    (Classical.choose_spec
      (exists_decomposition F W
        P w)).2).2
```

choose_spec

```
noncomputable def yComponent
  (P : CompletePolarization F W )
  (w : W) : W :=
  Classical.choose
  (Classical.choose_spec
    (exists_decomposition F W P
      w)).2
```

```
lemma decompose_by_components
  (P : CompletePolarization F W )
  (w : W) :
  w = xComponent F W P w
    + yComponent F W P w := by
  dsimp [xComponent, yComponent]
  exact
  (Classical.choose_spec
    (Classical.choose_spec
      (exists_decomposition F W
        P w)).2).2
```

Lean witness
choose_spec

choose_spec

```
noncomputable def yComponent
  (P : CompletePolarization F W )
  (w : W) : W :=
  Classical.choose
  (Classical.choose_spec
    (exists_decomposition F W P
      w)).2
```

```
lemma decompose_by_components
  (P : CompletePolarization F W )
  (w : W) :
  w = xComponent F W P w
    + yComponent F W P w := by
  dsimp [xComponent, yComponent]
  exact
  (Classical.choose_spec
    (Classical.choose_spec
      (exists_decomposition F W
        P w)).2).2
```

• x P.X

Lean witness
choose_spec

choose_spec

```
noncomputable def yComponent
  (P : CompletePolarization F W )
  (w : W) : W :=
  Classical.choose
    (Classical.choose_spec
      (exists_decomposition F W P
        w)).2
```

```
lemma decompose_by_components
  (P : CompletePolarization F W )
  (w : W) :
  w = xComponent F W P w
    + yComponent F W P w := by
  dsimp [xComponent, yComponent]
  exact
    (Classical.choose_spec
      (Classical.choose_spec
        (exists_decomposition F W
          P w)).2).2
```

- $x \quad P.X$
- $y \quad P.Y$

Lean witness
choose_spec

choose_spec

```
noncomputable def yComponent
  (P : CompletePolarization F W )
  (w : W) : W :=
  Classical.choose
    (Classical.choose_spec
      (exists_decomposition F W P
        w)).2
```

```
lemma decompose_by_components
  (P : CompletePolarization F W )
  (w : W) :
  w = xComponent F W P w
    + yComponent F W P w := by
  dsimp [xComponent, yComponent]
  exact
    (Classical.choose_spec
      (Classical.choose_spec
        (exists_decomposition F W
          P w)).2).2
```

- $x \quad P.X$
- $y \quad P.Y$
- y

Lean witness
choose_spec

choose_spec

```
noncomputable def yComponent
  (P : CompletePolarization F W )
  (w : W) : W :=
  Classical.choose
    (Classical.choose_spec
      (exists_decomposition F W P
        w)).2
```

```
lemma decompose_by_components
  (P : CompletePolarization F W )
  (w : W) :
  w = xComponent F W P w
    + yComponent F W P w := by
  dsimp [xComponent, yComponent]
  exact
    (Classical.choose_spec
      (Classical.choose_spec
        (exists_decomposition F W
          P w)).2).2
```

- $x \quad P.X$
- $y \quad P.Y$
- y
- $w = x + y$

Lean witness
choose_spec

Outline

universe

Lean

```
class CategoryStruct (obj :
  Type u)
  extends Quiver obj where
  id :   X : obj, Hom X X
  comp : {X Y Z : obj},
    (X → Y) → (Y → Z) → (X → Z)
class Category (obj : Type u)
  extends CategoryStruct
    obj where
  id_comp : {X Y} (f : X
    → Y),
    X → f = f
  comp_id : {X Y} (f : X
    → Y),
    f → Y = f
  assoc : f ∘ g ∘ h,
    (f ∘ g) ∘ h = f ∘ (g ∘ h)
```

universe

Lean

```
class CategoryStruct (obj :
  Type u)
  extends Quiver obj where
  id :   X : obj, Hom X X
  comp : {X Y Z : obj},
    (X → Y) → (Y → Z) → (X → Z)
class Category (obj : Type u)
  extends CategoryStruct
    obj where
  id_comp : {X Y} (f : X
    → Y),
    X → f = f
  comp_id : {X Y} (f : X
    → Y),
    f → Y = f
  assoc : f ∘ g ∘ h,
    (f ∘ g) ∘ h = f ∘ (g ∘ h)
```

universe

Lean

```

class CategoryStruct (obj :
  Type u)
  extends Quiver obj where
  id :   X : obj, Hom X X
  comp : {X Y Z : obj},
    (X → Y) → (Y → Z) → (X → Z)
class Category (obj : Type u)
  extends CategoryStruct
    obj where
  id_comp : {X Y} (f : X
    → Y),
    X → f = f
  comp_id : {X Y} (f : X
    → Y),
    f → Y = f
  assoc : f ∘ g ∘ h,
    (f ∘ g) ∘ h = f ∘ (g ∘ h)

```

universe

Lean

• Hom Quiver

```

class CategoryStruct (obj :
  Type u)
  extends Quiver obj where
  id :   X : obj, Hom X X
  comp : {X Y Z : obj},
    (X → Y) → (Y → Z) → (X → Z)
class Category (obj : Type u)
  extends CategoryStruct
    obj where
  id_comp : {X Y} (f : X
    → Y),
    X → f = f
  comp_id : {X Y} (f : X
    → Y),
    f → Y = f
  assoc : f ∘ g ∘ h,
    (f ∘ g) ∘ h = f ∘ (g ∘ h)

```

universe

Lean

• Hom Quiver

• $X \rightarrow id \rightarrow X$

```

class CategoryStruct (obj :
  Type u)
  extends Quiver obj where
id :   X : obj, Hom X X
comp : {X Y Z : obj},
      (X → Y) → (Y → Z) → (X → Z)
class Category (obj : Type u)
  extends CategoryStruct
    obj where
id_comp : {X Y} (f : X
  → Y),
  X → f = f
comp_id : {X Y} (f : X
  → Y),
  f → Y = f
assoc : f ∘ g ∘ h,
  (f ∘ g) ∘ h = f ∘ (g ∘ h)

```

universe

Lean

- Hom Quiver
- $X \rightarrow id \rightarrow X$
- $f \circ g$

```

class CategoryStruct (obj :
  Type u)
  extends Quiver obj where
id :   X : obj, Hom X X
comp : {X Y Z : obj},
      (X → Y) → (Y → Z) → (X → Z)
class Category (obj : Type u)
  extends CategoryStruct
    obj where
id_comp : {X Y} (f : X
  → Y),
  X → f = f
comp_id : {X Y} (f : X
  → Y),
  f → Y = f
assoc : f ∘ g ∘ h,
  (f ∘ g) ∘ h = f ∘ (g ∘ h)

```

universe

Lean

- Hom Quiver
- $X \rightarrow X$
- $f \circ g$
- instance


```

class CategoryStruct (obj :
  Type u)
  extends Quiver obj where
  id :   X : obj, Hom X X
  comp : {X Y Z : obj},
    (X → Y) → (Y → Z) → (X → Z)
class Category (obj : Type u)
  extends CategoryStruct
    obj where
  id_comp : {X Y} (f : X → Y),
    X → f = f
  comp_id : {X Y} (f : X → Y),
    f → Y = f
  assoc : f ∘ g ∘ h,
    (f ∘ g) ∘ h = f ∘ (g ∘ h)

```

universe

Lean

- Hom Quiver
- $X \rightarrow id X$
- $f \circ g$
- instance
-


```

structure Functor (C : Type u)
  [Category C]
  (D : Type u) [Category D]
  where
    obj : C → D
    map : {X Y : C},
      (X → Y) → (obj X → obj Y)
    map_id : X → obj X,
      map (X → X) = (obj X)
    map_comp : f → g,
      map (f → g) = map f → map g

```

```

structure Functor (C : Type u)
  [Category C]
  (D : Type u) [Category D]
  where
    obj : C → D
    map : {X Y : C},
      (X → Y) → (obj X → obj Y)
    map_id : X → obj X,
      map (X → X) = (obj X)
    map_comp : f → g,
      map (f → g) = map f → map g

```

```

structure Functor (C : Type u)
  [Category C]
  (D : Type u) [Category D]
  where
    obj : C → D
    map : {X Y : C},
      (X → Y) → (obj X → obj Y)
    map_id : f,
      map (f) = (obj f)
    map_comp : f g,
      map (f ∘ g) = map f ∘ map g

```

• $F.obj\ X$

```

structure Functor (C : Type u)
  [Category C]
  (D : Type u) [Category D]
  where
    obj : C → D
    map : {X Y : C},
      (X → Y) → (obj X → obj Y)
    map_id : f,
      map (f) = (obj f)
    map_comp : f g,
      map (f ∘ g) = map f ∘ map g

```

- $F.obj\ X$
- $F.map\ f$

```

structure Functor (C : Type u)
  [Category C]
  (D : Type u) [Category D]
  where
    obj : C → D
    map : {X Y : C},
      (X → Y) → (obj X → obj Y)
    map_id : f,
      map (f) = (obj f)
    map_comp : f g,
      map (f ∘ g) = map f ∘ map g

```

- $F.obj\ X$
- $F.map\ f$
- $F\ G$

```

structure Functor (C : Type u)
  [Category C]
  (D : Type u) [Category D]
  where
    obj : C → D
    map : {X Y : C},
      (X → Y) → (obj X → obj Y)
    map_id : {X : C},
      map (id X) = id (obj X)
    map_comp : {f g : C → C},
      map (f ∘ g) = map f ∘ map g

```

- $F.obj \ X$
- $F.map \ f$
- $F \ \mathcal{C} \ \mathcal{D}$
- $F : \mathcal{C} \rightarrow \mathcal{D}$

Mathlib morphism : F G

```
structure NatTrans (F G : C
  D) where
  app (X : C) : F.obj X
    G.obj X
  naturality X Y : C (f : X
    Y) :
    F.map f    app Y =
    app X    G.map f
```

Mathlib morphism : F G

```
structure NatTrans (F G : C
  D) where
  app (X : C) : F.obj X
    G.obj X
  naturality X Y : C (f : X
    Y) :
    F.map f    app Y =
    app X    G.map f
```

Mathlib morphism : F G

```

structure NatTrans (F G : C
  D) where
  app (X : C) : F.obj X
    G.obj X
  naturality X Y : C (f : X
    Y) :
    F.map f    app Y =
    app X    G.map f

```

• .app X

Mathlib morphism : F G

```

structure NatTrans (F G : C
  D) where
  app (X : C) : F.obj X
    G.obj X
  naturality X Y : C (f : X
    Y) :
    F.map f    app Y =
    app X    G.map f

```

• .app X

•

Mathlib morphism : F G

```

structure NatTrans (F G : C
  D) where
  app (X : C) : F.obj X
    G.obj X
  naturality X Y : C (f : X
    Y) :
    F.map f    app Y =
    app X    G.map f

```

• .app X

•

•

Mathlib morphism : F G

`ComplexShape.up``HomologicalComplex.d_comp_d`

```
structure HomologicalComplex
  (c : ComplexShape) where
  X :  $\rightarrow V$ 
  d :  $i \rightarrow j, X\ i \rightarrow X\ j$ 
  shape :  $i \rightarrow j$ ,
     $\neg c.Rel\ i\ j \rightarrow d\ i\ j = 0$ 
  d_comp_d' :  $i \rightarrow j \rightarrow k$ ,
     $c.Rel\ i\ j \rightarrow c.Rel\ j\ k \rightarrow$ 
       $d\ i\ j \circ d\ j\ k = 0$ 
```

ComplexShape.up

HomologicalComplex.d_comp_d


```
structure HomologicalComplex
  (c : ComplexShape) where
  X :  $\rightarrow V$ 
  d :  $i \rightarrow j, X\ i \rightarrow X\ j$ 
  shape :  $i \rightarrow j$ ,
     $\neg c.Rel\ i\ j \rightarrow d\ i\ j = 0$ 
  d_comp_d' :  $i \rightarrow j \rightarrow k$ ,
     $c.Rel\ i\ j \rightarrow c.Rel\ j\ k \rightarrow$ 
       $d\ i\ j \circ d\ j\ k = 0$ 
```

ComplexShape.up

HomologicalComplex.d_comp_d

```

structure HomologicalComplex
  (c : ComplexShape) where
  X :  $\rightarrow V$ 
  d :  $i \rightarrow j, X \ i \rightarrow X \ j$ 
  shape :  $i \rightarrow j$ ,
     $\neg c.Rel \ i \ j \rightarrow d \ i \ j = 0$ 
  d_comp_d' :  $i \rightarrow j \rightarrow k$ ,
     $c.Rel \ i \ j \rightarrow c.Rel \ j \ k \rightarrow$ 
       $d \ i \ j \circ d \ j \ k = 0$ 

```

• $X \ i \rightarrow i$

ComplexShape.up

HomologicalComplex.d_comp_d

```

structure HomologicalComplex
  (c : ComplexShape) where
  X :  $\rightarrow V$ 
  d :  $i \rightarrow j, X\ i \rightarrow X\ j$ 
  shape :  $i \rightarrow j$ ,
     $\neg c.Rel\ i\ j \rightarrow d\ i\ j = 0$ 
  d_comp_d' :  $i \rightarrow j \rightarrow k$ ,
     $c.Rel\ i\ j \rightarrow c.Rel\ j\ k \rightarrow$ 
       $d\ i\ j \circ d\ j\ k = 0$ 

```

- $X\ i$
- $d\ i\ j$

ComplexShape.up

HomologicalComplex.d_comp_d

```

structure HomologicalComplex
  (c : ComplexShape) where
  X :  $\rightarrow V$ 
  d :  $i \rightarrow j, X\ i \rightarrow X\ j$ 
  shape :  $i \rightarrow j$ ,
     $\neg c.Rel\ i\ j \rightarrow d\ i\ j = 0$ 
  d_comp_d' :  $i \rightarrow j \rightarrow k$ ,
     $c.Rel\ i\ j \rightarrow c.Rel\ j\ k \rightarrow$ 
       $d\ i\ j \circ d\ j\ k = 0$ 

```

- $X\ i$
- $d\ i\ j$
- $shape$

ComplexShape.up

HomologicalComplex.d_comp_d

```

structure HomologicalComplex
  (c : ComplexShape) where
  X :  $\rightarrow V$ 
  d :  $i \rightarrow j, X\ i \rightarrow X\ j$ 
  shape :  $i \rightarrow j$ ,
     $\neg c.Rel\ i\ j \rightarrow d\ i\ j = 0$ 
  d_comp_d' :  $i \rightarrow j \rightarrow k$ ,
     $c.Rel\ i\ j \rightarrow c.Rel\ j\ k \rightarrow$ 
       $d\ i\ j \circ d\ j\ k = 0$ 

```

- $X\ i \rightarrow i$
- $d\ i\ j$
- $shape$
- d_comp_d'

ComplexShape.up

HomologicalComplex.d_comp_d

Outline

Category instance

Category instance

```
open CategoryTheory
inductive OneObj where
  | star
instance : Category OneObj where
  Hom _ _ := PUnit
  id _ := PUnit.unit
  comp _ _ := PUnit.unit
  id_comp := by
    intro X Y f; cases f; rfl
  comp_id := by
    intro X Y f; cases f; rfl
  assoc := by
    intro W X Y Z f g h
    cases f; cases g; cases h; rfl
```


Category instance

```
open CategoryTheory
inductive OneObj where
  | star
instance : Category OneObj where
  Hom _ _ := PUnit
  id _ := PUnit.unit
  comp _ _ := PUnit.unit
  id_comp := by
    intro X Y f; cases f; rfl
  comp_id := by
    intro X Y f; cases f; rfl
  assoc := by
    intro W X Y Z f g h
    cases f; cases g; cases h; rfl
```

Category instance

```
open CategoryTheory
inductive OneObj where
  | star
instance : Category OneObj where
  Hom _ _ := PUnit
  id _ := PUnit.unit
  comp _ _ := PUnit.unit
  id_comp := by
    intro X Y f; cases f; rfl
  comp_id := by
    intro X Y f; cases f; rfl
  assoc := by
    intro W X Y Z f g h
    cases f; cases g; cases h; rfl
```

① Hom/id/comp

Category instance

```
open CategoryTheory
inductive OneObj where
  | star
instance : Category OneObj where
  Hom _ _ := PUnit
  id _ := PUnit.unit
  comp _ _ := PUnit.unit
  id_comp := by
    intro X Y f; cases f; rfl
  comp_id := by
    intro X Y f; cases f; rfl
  assoc := by
    intro W X Y Z f g h
    cases f; cases g; cases h; rfl
```

① Hom/id/comp

②

Category instance

```
open CategoryTheory
inductive OneObj where
  | star
instance : Category OneObj where
  Hom _ _ := PUnit
  id _ := PUnit.unit
  comp _ _ := PUnit.unit
  id_comp := by
    intro X Y f; cases f; rfl
  comp_id := by
    intro X Y f; cases f; rfl
  assoc := by
    intro W X Y Z f g h
    cases f; cases g; cases h; rfl
```

① Hom/id/comp

②

③

Category instance

```
open CategoryTheory
inductive OneObj where
  | star
instance : Category OneObj where
  Hom _ _ := PUnit
  id _ := PUnit.unit
  comp _ _ := PUnit.unit
  id_comp := by
    intro X Y f; cases f; rfl
  comp_id := by
    intro X Y f; cases f; rfl
  assoc := by
    intro W X Y Z f g h
    cases f; cases g; cases h; rfl
```

① Hom/id/comp

②

③

④

HomologicalComplex

Mathlib

HomologicalComplex

```
open CategoryTheory
open CategoryTheory.Limits
noncomputable def tinyZeroComplex :
  HomologicalComplex
    (Discrete PUnit)
    (ComplexShape.up ) where
X _ := Discrete.mk PUnit.unit
d _ _ := 0
shape := by
  intro i j hij; rfl
d_comp_d' := by
  intro i j k hij hjk; simp
```

Mathlib

HomologicalComplex

```
open CategoryTheory
open CategoryTheory.Limits
noncomputable def tinyZeroComplex :
  HomologicalComplex
    (Discrete PUnit)
    (ComplexShape.up ) where
X _ := Discrete.mk PUnit.unit
d _ _ := 0
shape := by
  intro i j hij; rfl
d_comp_d' := by
  intro i j k hij hjk; simp
```

Mathlib

HomologicalComplex

```
open CategoryTheory
open CategoryTheory.Limits
noncomputable def tinyZeroComplex :
  HomologicalComplex
    (Discrete PUnit)
    (ComplexShape.up ) where
X _ := Discrete.mk PUnit.unit
d _ _ := 0
shape := by
  intro i j hij; rfl
d_comp_d' := by
  intro i j k hij hjk; simp
```

Mathlib

- Discrete PUnit

HomologicalComplex

```
open CategoryTheory
open CategoryTheory.Limits
noncomputable def tinyZeroComplex :
  HomologicalComplex
    (Discrete PUnit)
    (ComplexShape.up ) where
X _ := Discrete.mk PUnit.unit
d _ _ := 0
shape := by
  intro i j hij; rfl
d_comp_d' := by
  intro i j k hij hjk; simp
```

Mathlib

- Discrete PUnit
- 0

HomologicalComplex

```
open CategoryTheory
open CategoryTheory.Limits
noncomputable def tinyZeroComplex :
  HomologicalComplex
    (Discrete PUnit)
    (ComplexShape.up ) where
X _ := Discrete.mk PUnit.unit
d _ _ := 0
shape := by
  intro i j hij; rfl
d_comp_d' := by
  intro i j k hij hjk; simp
```

Mathlib

- Discrete PUnit
- 0
- shape

HomologicalComplex

```
open CategoryTheory
open CategoryTheory.Limits
noncomputable def tinyZeroComplex :
  HomologicalComplex
    (Discrete PUnit)
    (ComplexShape.up ) where
X _ := Discrete.mk PUnit.unit
d _ _ := 0
shape := by
  intro i j hij; rfl
d_comp_d' := by
  intro i j k hij hjk; simp
```

Mathlib

- Discrete PUnit
- 0
- shape
- simp

Outline

instance

instance

instance

instance

instance

- Type

instance

instance

- Type
-

instance

instance

- Type
-
- Group G , Field K

instance

instance

- Type
-
- Group G , Field K
-

instance

- Type
-
- Group G , Field K
-

instance

instance

instance

- Type
-
- Group G , Field K
-

instance

- Prop

instance

instance

- Type
-
- Group G , Field K
-

instance

- Prop
-

instance

instance

- Type
-
- Group G , Field K
-

instance

- Prop
-
- Fact P , [Decidable P]

instance

instance

- Type
-
- Group G , Field K
-

instance

- Prop
-
- Fact P , [Decidable P]
-


```
-- Fact P wraps a proof of P as an
  instance
-- class Fact (P : Prop) : Prop
  where
--   out : P

-- Usage: bring a hypothesis into
  instance search
example (P : Prop) [Fact P] : P :=
  Fact.out

-- haveI temporarily introduces an
  instance
example (n : ) (h : n 0) : 0 < n
  := by
  haveI : Fact (n 0) := h
  exact Nat.pos_of_ne_zero Fact.out
```

```
-- Fact P wraps a proof of P as an
  instance
-- class Fact (P : Prop) : Prop
  where
--   out : P

-- Usage: bring a hypothesis into
  instance search
example (P : Prop) [Fact P] : P :=
  Fact.out

-- haveI temporarily introduces an
  instance
example (n : ) (h : n 0) : 0 < n
  := by
  haveI : Fact (n 0) := h
  exact Nat.pos_of_ne_zero Fact.out
```

```

-- Fact P wraps a proof of P as an
  instance
-- class Fact (P : Prop) : Prop
  where
--   out : P

-- Usage: bring a hypothesis into
  instance search
example (P : Prop) [Fact P] : P :=
  Fact.out

-- haveI temporarily introduces an
  instance
example (n : ) (h : n 0) : 0 < n
  := by
  haveI : Fact (n 0) := h
  exact Nat.pos_of_ne_zero Fact.out

```

① Fact P P

```

-- Fact P wraps a proof of P as an
  instance
-- class Fact (P : Prop) : Prop
  where
--   out : P

-- Usage: bring a hypothesis into
  instance search
example (P : Prop) [Fact P] : P :=
  Fact.out

-- haveI temporarily introduces an
  instance
example (n : ) (h : n 0) : 0 < n
  := by
  haveI : Fact (n 0) := h
  exact Nat.pos_of_ne_zero Fact.out

```

① Fact P P

② Fact.out

```

-- Fact P wraps a proof of P as an
  instance
-- class Fact (P : Prop) : Prop
  where
--   out : P

-- Usage: bring a hypothesis into
  instance search
example (P : Prop) [Fact P] : P :=
  Fact.out

-- haveI temporarily introduces an
  instance
example (n : ) (h : n 0) : 0 < n
  := by
  haveI : Fact (n 0) := h
  exact Nat.pos_of_ne_zero Fact.out

```

- ① Fact P P
- ② Fact.out
- ③ [Fact ...]

```

-- Fact P wraps a proof of P as an
  instance
-- class Fact (P : Prop) : Prop
  where
--   out : P

-- Usage: bring a hypothesis into
  instance search
example (P : Prop) [Fact P] : P :=
  Fact.out

-- haveI temporarily introduces an
  instance
example (n : ) (h : n 0) : 0 < n
  := by
  haveI : Fact (n 0) := h
  exact Nat.pos_of_ne_zero Fact.out

```

- ① Fact P P
- ② Fact.out
- ③ [Fact ...]
- ④ haveI

class

class

```
class IsDivisibleBy (n k : ) :  
  Prop where  
  witness : k n  
  
-- Use an instance implicitly  
example (n k : ) [IsDivisibleBy n  
  k] :  
  k n :=  
  IsDivisibleBy.witness  
  
-- inferInstance lets Lean search  
  for the proof  
example (n k : ) [h :  
  IsDivisibleBy n k] :  
  k n :=  
  (inferInstance : IsDivisibleBy n  
    k).witness
```

class

```
class IsDivisibleBy (n k : ) :  
  Prop where  
  witness : k  n  
  
-- Use an instance implicitly  
example (n k : ) [IsDivisibleBy n  
  k] :  
  k  n :=  
  IsDivisibleBy.witness  
  
-- inferInstance lets Lean search  
  for the proof  
example (n k : ) [h :  
  IsDivisibleBy n k] :  
  k  n :=  
  (inferInstance : IsDivisibleBy n  
    k).witness
```

class

```
class IsDivisibleBy (n k : ) :  
  Prop where  
  witness : k n  
  
-- Use an instance implicitly  
example (n k : ) [IsDivisibleBy n  
  k] :  
  k n :=  
  IsDivisibleBy.witness  
  
-- inferInstance lets Lean search  
  for the proof  
example (n k : ) [h :  
  IsDivisibleBy n k] :  
  k n :=  
  (inferInstance : IsDivisibleBy n  
    k).witness
```

class

```
class IsDivisibleBy (n k : ) :  
  Prop where  
  witness : k  n  
  
-- Use an instance implicitly  
example (n k : ) [IsDivisibleBy n  
  k] :  
  k  n :=  
  IsDivisibleBy.witness  
  
-- inferInstance lets Lean search  
  for the proof  
example (n k : ) [h :  
  IsDivisibleBy n k] :  
  k  n :=  
  (inferInstance : IsDivisibleBy n  
    k).witness
```

class

```
class IsDivisibleBy (n k : ) :
  Prop where
  witness : k  n

-- Use an instance implicitly
example (n k : ) [IsDivisibleBy n
  k] :
  k  n :=
  IsDivisibleBy.witness

-- inferInstance lets Lean search
  for the proof
example (n k : ) [h :
  IsDivisibleBy n k] :
  k  n :=
  (inferInstance : IsDivisibleBy n
    k).witness
```

-
-
- inferInstance
Lean

class

```
class IsDivisibleBy (n k : ) :  
  Prop where  
  witness : k n  
  
-- Use an instance implicitly  
example (n k : ) [IsDivisibleBy n  
  k] :  
  k n :=  
  IsDivisibleBy.witness  
  
-- inferInstance lets Lean search  
  for the proof  
example (n k : ) [h :  
  IsDivisibleBy n k] :  
  k n :=  
  (inferInstance : IsDivisibleBy n  
    k).witness
```

-
-
- inferInstance
Lean
-

Outline

instance search

#synth	infer_instance	haveI
Lean	proof	

```
-- Check that an instance exists
#synth Group Sign

-- Use it inside a proof
example : Group Sign := infer_instance

-- Temporarily introduce an instance
example (G : Type*) [Group G]
  [h : CommGroup G] (a b : G) :
    a * b = b * a := by
  haveI : CommGroup G := h
  exact mul_comm a b
```


—

```

-- mul_assoc needs only Semigroup
#check @mul_assoc
--   {} [Semigroup ] (a b c : ),
--   a * b * c = a * (b * c)

-- inv_mul_cancel needs Group
#check @inv_mul_cancel
--   {G} [Group G] (a : G),
--   a1 * a = 1

-- group tactic works for any Group
example {G : Type*} [Group G]
  (a b : G) :
    (a * b)1 = b1 * a1 := by
  group

```

```

-- mul_assoc needs only Semigroup
#check @mul_assoc
--   {} [Semigroup ] (a b c : ),
--   a * b * c = a * (b * c)

-- inv_mul_cancel needs Group
#check @inv_mul_cancel
--   {G} [Group G] (a : G),
--   a1 * a = 1

-- group tactic works for any Group
example {G : Type*} [Group G]
  (a b : G) :
  (a * b)1 = b1 * a1 := by
  group

```

```

-- mul_assoc needs only Semigroup
#check @mul_assoc
--   {} [Semigroup ] (a b c : ),
--   a * b * c = a * (b * c)

-- inv_mul_cancel needs Group
#check @inv_mul_cancel
--   {G} [Group G] (a : G),
--   a1 * a = 1

-- group tactic works for any Group
example {G : Type*} [Group G]
  (a b : G) :
  (a * b)1 = b1 * a1 := by
  group

```

```

-- mul_assoc needs only Semigroup
#check @mul_assoc
--   {} [Semigroup ] (a b c : ),
--   a * b * c = a * (b * c)

-- inv_mul_cancel needs Group
#check @inv_mul_cancel
--   {G} [Group G] (a : G),
--   a1 * a = 1

-- group tactic works for any Group
example {G : Type*} [Group G]
  (a b : G) :
  (a * b)1 = b1 * a1 := by
  group

```

-
- [Group G]
[CommGroup G]

```

-- mul_assoc needs only Semigroup
#check @mul_assoc
--   {} [Semigroup ] (a b c : ),
--   a * b * c = a * (b * c)

-- inv_mul_cancel needs Group
#check @inv_mul_cancel
--   {G} [Group G] (a : G),
--   a1 * a = 1

-- group tactic works for any Group
example {G : Type*} [Group G]
  (a b : G) :
  (a * b)1 = b1 * a1 := by
  group

```

-
- [Group G]
[CommGroup G]
- AI

```

-- mul_assoc needs only Semigroup
#check @mul_assoc
--   {} [Semigroup ] (a b c : ),
--   a * b * c = a * (b * c)

-- inv_mul_cancel needs Group
#check @inv_mul_cancel
--   {G} [Group G] (a : G),
--   a1 * a = 1

-- group tactic works for any Group
example {G : Type*} [Group G]
  (a b : G) :
  (a * b)1 = b1 * a1 := by
  group

```

-
- [Group G]
[CommGroup G]
- AI
- #check

— Equiv instance

instance Mathlib

— Equiv instance

```
-- Transfer CommRing along an
   equivalence
def equivProd : PairInt    Int × Int
  := ...
instance : CommRing PairInt :=
  equivProd.commRing

-- Transfer Field along an
   equivalence
def equivRat : WrappedQ    := ...
instance : Field WrappedQ :=
  equivRat.field
```

instance Mathlib

— Equiv instance

```
-- Transfer CommRing along an
   equivalence
def equivProd : PairInt    Int × Int
  := ...
instance : CommRing PairInt :=
  equivProd.commRing

-- Transfer Field along an
   equivalence
def equivRat : WrappedQ    := ...
instance : Field WrappedQ :=
  equivRat.field
```

instance Mathlib

— Equiv instance

```
-- Transfer CommRing along an
   equivalence
def equivProd : PairInt    Int × Int
  := ...
instance : CommRing PairInt :=
  equivProd.commRing

-- Transfer Field along an
   equivalence
def equivRat : WrappedQ    := ...
instance : Field WrappedQ :=
  equivRat.field
```

instance Mathlib

— Equiv instance

```
-- Transfer CommRing along an
   equivalence
def equivProd : PairInt    Int × Int
  := ...
instance : CommRing PairInt :=
  equivProd.commRing

-- Transfer Field along an
   equivalence
def equivRat : WrappedQ    := ...
instance : Field WrappedQ :=
  equivRat.field
```

instance Mathlib



equivalence

— Equiv instance

```
-- Transfer CommRing along an
   equivalence
def equivProd : PairInt    Int × Int
  := ...
instance : CommRing PairInt :=
  equivProd.commRing

-- Transfer Field along an
   equivalence
def equivRat : WrappedQ    := ...
instance : Field WrappedQ :=
  equivRat.field
```

instance Mathlib

-
- equivalence
- wrapper subtype

— Equiv instance

```
-- Transfer CommRing along an
   equivalence
def equivProd : PairInt    Int × Int
  := ...
instance : CommRing PairInt :=
  equivProd.commRing

-- Transfer Field along an
   equivalence
def equivRat : WrappedQ    := ...
instance : Field WrappedQ :=
  equivRat.field
```

instance Mathlib

-
- equivalence
- wrapper subtype
- AI Lean

Outline

statement

statement

```
--      Missing hypothesis: what if  $x$ 
      = 0?
example (K : Type*) [Field K] (x :
  K) :
  x / x = 1 := by
    field_simp          -- fails!

--      Add the necessary assumption
example (K : Type*) [Field K]
  (x : K) (hx : x ≠ 0) :
  x / x = 1 := by
    field_simp [hx]
```

statement

```
--      Missing hypothesis: what if  $x$ 
      = 0?
example (K : Type*) [Field K] (x :
  K) :
  x / x = 1 := by
    field_simp          -- fails!

--      Add the necessary assumption
example (K : Type*) [Field K]
  (x : K) (hx : x ≠ 0) :
  x / x = 1 := by
    field_simp [hx]
```

statement

```
--    Missing hypothesis: what if x  
      = 0?
```

```
example (K : Type*) [Field K] (x :  
  K) :  
  x / x = 1 := by  
  field_simp          -- fails!
```

```
--    Add the necessary assumption
```

```
example (K : Type*) [Field K]  
  (x : K) (hx : x ≠ 0) :  
  x / x = 1 := by  
  field_simp [hx]
```

$$x = 0$$

statement

```
--      Missing hypothesis: what if  $x$ 
      = 0?
example (K : Type*) [Field K] (x :
  K) :
  x / x = 1 := by
    field_simp          -- fails!

--      Add the necessary assumption
example (K : Type*) [Field K]
  (x : K) (hx : x ≠ 0) :
  x / x = 1 := by
    field_simp [hx]
```

- $x = 0$
- Lean

statement

```
--      Missing hypothesis: what if  $x$ 
      = 0?
example (K : Type*) [Field K] (x :
  K) :
  x / x = 1 := by
    field_simp          -- fails!

--      Add the necessary assumption
example (K : Type*) [Field K]
  (x : K) (hx : x ≠ 0) :
  x / x = 1 := by
    field_simp [hx]
```

- $x = 0$
- Lean
- tactic

statement

```
--      Missing hypothesis: what if  $x$ 
      = 0?
example (K : Type*) [Field K] (x :
  K) :
  x / x = 1 := by
    field_simp          -- fails!

--      Add the necessary assumption
example (K : Type*) [Field K]
  (x : K) (hx : x ≠ 0) :
  x / x = 1 := by
    field_simp [hx]
```

- $x = 0$
- Lean
- tactic
- statement proof


```

--      False in a non-commutative
      group!
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) 1 = a1 * b1 := by
group      -- fails: not provable

--      Correct for any Group
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) 1 = b1 * a1 := by
group

--      Commutative version needs
      CommGroup
example (G : Type*) [CommGroup G]
  (a b : G) :
  (a * b) 1 = a1 * b1 := by
group

```



```

--      False in a non-commutative
      group!
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) 1 = a1 * b1 := by
group      -- fails: not provable

--      Correct for any Group
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) 1 = b1 * a1 := by
group

--      Commutative version needs
      CommGroup
example (G : Type*) [CommGroup G]
  (a b : G) :
  (a * b) 1 = a1 * b1 := by
group

```

```

--      False in a non-commutative
      group!
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) 1 = a1 * b1 := by
group      -- fails: not provable

```

```

--      Correct for any Group
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) 1 = b1 * a1 := by
group

```

```

--      Commutative version needs
      CommGroup
example (G : Type*) [CommGroup G]
  (a b : G) :
  (a * b) 1 = a1 * b1 := by
group

```

```
--      False in a non-commutative
      group!
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) 1 = a1 * b1 := by
group      -- fails: not provable
```



• group

```
--      Correct for any Group
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) 1 = b1 * a1 := by
group
```

```
--      Commutative version needs
      CommGroup
example (G : Type*) [CommGroup G]
  (a b : G) :
  (a * b) 1 = a1 * b1 := by
group
```

```

--      False in a non-commutative
      group!
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) 1 = a1 * b1 := by
group      -- fails: not provable

```

```

--      Correct for any Group
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) 1 = b1 * a1 := by
group

```

```

--      Commutative version needs
      CommGroup
example (G : Type*) [CommGroup G]
  (a b : G) :
  (a * b) 1 = a1 * b1 := by
group

```

-
- group
- [CommGroup G]

```
--      False in a non-commutative
      group!
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) 1 = a1 * b1 := by
  group      -- fails: not provable
```

```
--      Correct for any Group
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) 1 = b1 * a1 := by
  group
```

```
--      Commutative version needs
      CommGroup
example (G : Type*) [CommGroup G]
  (a b : G) :
  (a * b) 1 = a1 * b1 := by
  group
```

-
- group
- [CommGroup G]
-


```

--      This theorem name does not
      exist
example (n : ) :
  (Finset.range n).card = n := by
    exact Finset.card_range_eq_n --
      Error!

--      Check the real API first
#check Finset.card_range
-- Finset.card_range :
--      (n : ), (Finset.range
      n).card = n

example (n : ) :
  (Finset.range n).card = n :=
    Finset.card_range n

```

```

--      This theorem name does not
      exist
example (n : ) :
  (Finset.range n).card = n := by
    exact Finset.card_range_eq_n --
      Error!

--      Check the real API first
#check Finset.card_range
-- Finset.card_range :
--      (n : ), (Finset.range
      n).card = n

example (n : ) :
  (Finset.range n).card = n :=
    Finset.card_range n

```



```

--      This theorem name does not
      exist
example (n : ) :
  (Finset.range n).card = n := by
    exact Finset.card_range_eq_n --
      Error!

--      Check the real API first
#check Finset.card_range
-- Finset.card_range :
--      (n : ), (Finset.range
      n).card = n

example (n : ) :
  (Finset.range n).card = n :=
    Finset.card_range n

```

```

--      This theorem name does not
      exist
example (n : ) :
  (Finset.range n).card = n := by
    exact Finset.card_range_eq_n --
      Error!

--      Check the real API first
#check Finset.card_range
-- Finset.card_range :
--      (n : ), (Finset.range
      n).card = n

example (n : ) :
  (Finset.range n).card = n :=
    Finset.card_range n

```

API

```
--      This theorem name does not
      exist
example (n : ) :
  (Finset.range n).card = n := by
    exact Finset.card_range_eq_n --
      Error!

--      Check the real API first
#check Finset.card_range
-- Finset.card_range :
--      (n : ), (Finset.range
      n).card = n

example (n : ) :
  (Finset.range n).card = n :=
    Finset.card_range n
```

-
- API
- AI #check

```

--      This theorem name does not
      exist
example (n : ) :
  (Finset.range n).card = n := by
    exact Finset.card_range_eq_n --
      Error!

--      Check the real API first
#check Finset.card_range
-- Finset.card_range :
--      (n : ), (Finset.range
      n).card = n

example (n : ) :
  (Finset.range n).card = n :=
    Finset.card_range n

```

-
- API
- AI #check
- LeanSearch / Loogle

decide

decide

```
--    decide requires [Decidable P]
example (P : Prop) : P  → P := by
  decide      -- Error: no Decidable
               instance

--    Constructive: add [Decidable
P]
example (P : Prop) [Decidable P] :
  P  → P := by
  by_cases h : P
  · exact Or.inl h
  · exact Or.inr h

--    Classical: use em
example (P : Prop) : P  → P := by
  classical; exact em P
```

decide

```
--    decide requires [Decidable P]
example (P : Prop) : P  → P := by
  decide      -- Error: no Decidable
               instance

--    Constructive: add [Decidable
P]
example (P : Prop) [Decidable P] :
  P  → P := by
  by_cases h : P
  · exact Or.inl h
  · exact Or.inr h

--    Classical: use em
example (P : Prop) : P  → P := by
  classical; exact em P
```

decide

```
--    decide requires [Decidable P]
example (P : Prop) : P  ¬ P := by
  decide    -- Error: no Decidable
             instance

--    Constructive: add [Decidable
P]
example (P : Prop) [Decidable P] :
  P  ¬ P := by
  by_cases h : P
  · exact Or.inl h
  · exact Or.inr h

--    Classical: use em
example (P : Prop) : P  ¬ P := by
  classical; exact em P
```

❶ decide
[Decidable P]

decide

```
--      decide requires [Decidable P]
example (P : Prop) : P  ¬ P := by
  decide      -- Error: no Decidable
              instance

--      Constructive: add [Decidable
P]
example (P : Prop) [Decidable P] :
  P  ¬ P := by
  by_cases h : P
  · exact Or.inl h
  · exact Or.inr h

--      Classical: use em
example (P : Prop) : P  ¬ P := by
  classical; exact em P
```

- ① decide
[Decidable P]
- ② decide

decide

```
--      decide requires [Decidable P]
example (P : Prop) : P  ¬ P := by
  decide      -- Error: no Decidable
               instance

--      Constructive: add [Decidable
P]
example (P : Prop) [Decidable P] :
  P  ¬ P := by
  by_cases h : P
  · exact Or.inl h
  · exact Or.inr h

--      Classical: use em
example (P : Prop) : P  ¬ P := by
  classical; exact em P
```

- ① decide
[Decidable P]
- ② decide
- ③ classical

decide

```
--    decide requires [Decidable P]
example (P : Prop) : P  → P := by
  decide    -- Error: no Decidable
             instance

--    Constructive: add [Decidable
P]
example (P : Prop) [Decidable P] :
  P  → P := by
  by_cases h : P
  · exact Or.inl h
  · exact Or.inr h

--    Classical: use em
example (P : Prop) : P  → P := by
  classical; exact em P
```

- ① decide
[Decidable P]
- ② decide
- ③ classical
- ④ AI

Outline

	AI	Lean
Statement	typeclass	
Search	#check	theorem
Proof	proof	lemma
Review		simp
AI	Lean kernel	

Prompt

Lean 4 / Mathlib
theorem statement

1. typeclass
- 2.
3. #check
4. theorem
5. statement

→ →

	$x / x = 1$	$x \cdot 0$	AI
	Group	CommGroup	#check
API	theorem		#check
			Mathlib
	tactic		lemma proof

Outline

Lean

Lean

- theorem statement

Lean

- theorem statement
- proof term

Lean

- theorem statement
- proof term
- instance

Lean

- theorem statement
- proof term
- instance
-

Lean

- theorem statement
- proof term
- instance
-

Lean

Lean

- theorem statement
- proof term
- instance
-

Lean

-

Lean

- theorem statement
- proof term
- instance
-

Lean

-
- Mathlib

Lean

- theorem statement
- proof term
- instance
-

Lean

-
- Mathlib
-

Lean

- theorem statement
- proof term
- instance
-

Lean

-
- Mathlib
-
-

Outline

Finset α

Finset α



Finset α

-
- card

Finset α

-
- card
- range n $\{0, \dots, n - 1\}$

Finset α

-
- `card`
- `range n` $\{0, \dots, n - 1\}$
- `filter`

Finset α

-
- `card`
- `range n` $\{0, \dots, n - 1\}$
- `filter`

Finset α

- - `card`
 - `range n` $\{0, \dots, n - 1\}$
 - `filter`
- `#check`
`Finset.card_range`

Finset α

-
- `card`
- `range n` $\{0, \dots, n - 1\}$
- `filter`
- `#check`
`Finset.card_range`
- `simp`

Finset α

-
- `card`
- `range n` $\{0, \dots, n - 1\}$
- `filter`
- `#check`
`Finset.card_range`
- `simp`
- `native_decide`

Finset α

-
- `card`
- `range n` $\{0, \dots, n - 1\}$
- `filter`
- `#check`
`Finset.card_range`
- `simp`
- `native_decide`
- `sum_range_succ`

Finset

```
open BigOperators Finset

-- card of range n is n (general)
#check Finset.card_range
-- (n : ℕ), (Finset.range n).card
   = n
example (n : ℕ) :
  (Finset.range n).card = n :=
  Finset.card_range n

-- constant sum over range
  (general)
example (n : ℕ) :
  ∑ i in Finset.range n, (1 : ℕ) =
    n := by
  simp

-- counting evens in {0,...,2k-1}
  (general)
example (k : ℕ) :
  ((Finset.range (2 * k)).filter
    fun n => n % 2 = 0).card = k
    := by
  simp [Finset.card_filter]
```

Finset

```
open BigOperators Finset

-- card of range n is n (general)
#check Finset.card_range
-- (n : ℕ), (Finset.range n).card
   = n
example (n : ℕ) :
  (Finset.range n).card = n :=
  Finset.card_range n

-- constant sum over range
  (general)
example (n : ℕ) :
  ∑_i Finset.range n, (1 : ℕ) =
    n := by
  simp

-- counting evens in {0,...,2k-1}
  (general)
example (k : ℕ) :
  ((Finset.range (2 * k)).filter
    fun n => n % 2 = 0).card = k
    := by
  simp [Finset.card_filter]
```

`open BigOperators Finset`

```
-- card of range n is n (general)
#check Finset.card_range
-- (n : ℕ), (Finset.range n).card
   = n
```

```
example (n : ℕ) :
  (Finset.range n).card = n :=
  Finset.card_range n
```

```
-- constant sum over range
   (general)
```

```
example (n : ℕ) :
  ∑ i in Finset.range n, (1 : ℕ) =
  n := by
  simp
```

```
-- counting evens in  $\{0, \dots, 2k-1\}$ 
   (general)
```

```
example (k : ℕ) :
  ((Finset.range (2 * k)).filter
   fun n => n % 2 = 0).card = k
   := by
  simp [Finset.card_filter]
```

• API

Finset

```
open BigOperators Finset
```

```
-- card of range n is n (general)
#check Finset.card_range
-- (n : ℕ), (Finset.range n).card
   = n
```

```
example (n : ℕ) :
  (Finset.range n).card = n :=
  Finset.card_range n
```

```
-- constant sum over range
   (general)
```

```
example (n : ℕ) :
  ∑ i in Finset.range n, (1 : ℕ) =
    n := by
  simp
```

```
-- counting evens in  $\{0, \dots, 2k-1\}$ 
   (general)
```

```
example (k : ℕ) :
  ((Finset.range (2 * k)).filter
    fun n => n % 2 = 0).card = k
    := by
  simp [Finset.card_filter]
```

- API

-

Finset

```
open BigOperators Finset
```

```
-- card of range n is n (general)
#check Finset.card_range
-- (n : ℕ), (Finset.range n).card
   = n
```

```
example (n : ℕ) :
  (Finset.range n).card = n :=
  Finset.card_range n
```

```
-- constant sum over range
   (general)
```

```
example (n : ℕ) :
  ∑ i in Finset.range n, (1 : ℕ) =
  n := by
  simp
```

```
-- counting evens in  $\{0, \dots, 2k-1\}$ 
   (general)
```

```
example (k : ℕ) :
  ((Finset.range (2 * k)).filter
   fun n => n % 2 = 0).card = k
   := by
  simp [Finset.card_filter]
```

- API

-

-

```
open BigOperators Finset
```

```
-- card of range n is n (general)
#check Finset.card_range
-- (n : ℕ), (Finset.range n).card
   = n
```

```
example (n : ℕ) :
  (Finset.range n).card = n :=
  Finset.card_range n
```

```
-- constant sum over range
   (general)
```

```
example (n : ℕ) :
  ∑ i in Finset.range n, (1 : ℕ) =
    n := by
  simp
```

```
-- counting evens in  $\{0, \dots, 2k-1\}$ 
   (general)
```

```
example (k : ℕ) :
  ((Finset.range (2 * k)).filter
    fun n => n % 2 = 0).card = k
    := by
  simp [Finset.card_filter]
```

- API

-

-

-


```
-- General theorem (Gauss sum)
theorem sum_range_id (n : ℕ) :
  ∑ i in Finset.range (n + 1), i =
    n * (n + 1) / 2 := by
  induction n with
  | zero => simp
  | succ n ih =>
    rw [Finset.sum_range_succ, +
      ih]
    ring

-- Verify the identity holds
#check Finset.sum_range_id
-- ∑ i in Finset.range n, i = n *
  (n-1) / 2
```



```

-- General theorem (Gauss sum)
theorem sum_range_id (n : ℕ) :
  ∑ i in Finset.range (n + 1), i =
    n * (n + 1) / 2 := by
  induction n with
  | zero => simp
  | succ n ih =>
    rw [Finset.sum_range_succ, +
        ih]
    ring

-- Verify the identity holds
#check Finset.sum_range_id
-- ∑ i in Finset.range n, i = n *
  (n-1) / 2

```

Finset

tactic

```

-- General theorem (Gauss sum)
theorem sum_range_id (n : ℕ) :
  ∑ i in Finset.range (n + 1), i =
    n * (n + 1) / 2 := by
  induction n with
  | zero => simp
  | succ n ih =>
    rw [Finset.sum_range_succ, +
        ih]
    ring

-- Verify the identity holds
#check Finset.sum_range_id
-- ∑ i in Finset.range n, i = n *
  (n-1) / 2

```

① sum_range_succ

Finset

tactic

```

-- General theorem (Gauss sum)
theorem sum_range_id (n : ℕ) :
  ∑ i in Finset.range (n + 1), i =
    n * (n + 1) / 2 := by
  induction n with
  | zero => simp
  | succ n ih =>
    rw [Finset.sum_range_succ, +
        ih]
    ring

-- Verify the identity holds
#check Finset.sum_range_id
-- ∑ i in Finset.range n, i = n *
  (n-1) / 2

```

① sum_range_succ

②

Finset

tactic

```

-- General theorem (Gauss sum)
theorem sum_range_id (n : ℕ) :
  ∑ i in Finset.range (n + 1), i =
    n * (n + 1) / 2 := by
  induction n with
  | zero => simp
  | succ n ih =>
    rw [Finset.sum_range_succ, +
        ih]
    ring

-- Verify the identity holds
#check Finset.sum_range_id
-- ∑ i in Finset.range n, i = n *
  (n-1) / 2

```

① sum_range_succ

②

③ ring

Finset

tactic

```

-- General theorem (Gauss sum)
theorem sum_range_id (n : ℕ) :
  ∑ i in Finset.range (n + 1), i =
    n * (n + 1) / 2 := by
  induction n with
  | zero => simp
  | succ n ih =>
    rw [Finset.sum_range_succ, +
        ih]
    ring

-- Verify the identity holds
#check Finset.sum_range_id
-- ∑ i in Finset.range n, i = n *
  (n-1) / 2

```

① sum_range_succ

②

③ ring

④

Finset

tactic

Outline

— SimpleGraph = +

Lean

```
G : SimpleGraph V V
G.Adj u v          u v
symm
loopless            G.Adj v v
V = Fin n          fin_cases
```

```

def path3 : SimpleGraph (Fin 3)
  where
Adj i j :=
  (i = 0    j = 1)  (i = 1    j =
    0)
  (i = 1    j = 2)  (i = 2    j = 1)
symm := by
  intro i j h
  fin_cases i <;> fin_cases j <;>
  simp_all
loopless := by
  intro i h
  fin_cases i <;> simp_all

```

```

def path3 : SimpleGraph (Fin 3)
  where
Adj i j :=
  (i = 0    j = 1)  (i = 1    j =
    0)
  (i = 1    j = 2)  (i = 2    j = 1)
symm := by
  intro i j h
  fin_cases i <;> fin_cases j <;>
  simp_all
loopless := by
  intro i h
  fin_cases i <;> simp_all

```

```

def path3 : SimpleGraph (Fin 3)
  where
Adj i j :=
  (i = 0    j = 1)  (i = 1    j =
    0)
  (i = 1    j = 2)  (i = 2    j = 1)
symm := by
  intro i j h
  fin_cases i <;> fin_cases j <;>
  simp_all
loopless := by
  intro i h
  fin_cases i <;> simp_all

```

• 0, 1, 2

```

def path3 : SimpleGraph (Fin 3)
  where
Adj i j :=
  (i = 0    j = 1)  (i = 1    j =
    0)
  (i = 1    j = 2)  (i = 2    j = 1)
symm := by
  intro i j h
  fin_cases i <.> fin_cases j <.>
  simp_all
loopless := by
  intro i h
  fin_cases i <.> simp_all

```

- 0, 1, 2
- 0--1 1--2

```

def path3 : SimpleGraph (Fin 3)
  where
Adj i j :=
  (i = 0    j = 1)  (i = 1    j =
    0)
  (i = 1    j = 2)  (i = 2    j = 1)
symm := by
  intro i j h
  fin_cases i <;> fin_cases j <;>
  simp_all
loopless := by
  intro i h
  fin_cases i <;> simp_all

```

- 0, 1, 2
- 0--1 1--2
- symm

```

def path3 : SimpleGraph (Fin 3)
  where
Adj i j :=
  (i = 0    j = 1)  (i = 1    j =
    0)
  (i = 1    j = 2)  (i = 2    j = 1)
symm := by
  intro i j h
  fin_cases i <;> fin_cases j <;>
  simp_all
loopless := by
  intro i h
  fin_cases i <;> simp_all

```

- 0, 1, 2
- 0--1 1--2
- symm
- loopless


```
-- General: adjacency implies
distinct vertices
example (V : Type*) (G :
  SimpleGraph V)
  (u v : V) (h : G.Adj u v) : u
    v :=
  G.ne_of_adj h

-- Concrete: path3 adjacency
example : path3.Adj 0 1 := by simp
  [path3]
example : path3.Adj 1 2 := by simp
  [path3]
example : ¬ path3.Adj 0 2 := by
  simp [path3]

-- General: no vertex is adjacent
to itself
example (V : Type*) (G :
  SimpleGraph V)
  (v : V) : ¬ G.Adj v v :=
  G.loopless v
```



```
-- General: adjacency implies
distinct vertices
example (V : Type*) (G :
  SimpleGraph V)
  (u v : V) (h : G.Adj u v) : u
    v :=
  G.ne_of_adj h

-- Concrete: path3 adjacency
example : path3.Adj 0 1 := by simp
  [path3]
example : path3.Adj 1 2 := by simp
  [path3]
example : ¬ path3.Adj 0 2 := by
  simp [path3]

-- General: no vertex is adjacent
to itself
example (V : Type*) (G :
  SimpleGraph V)
  (v : V) : ¬ G.Adj v v :=
  G.loopless v
```

```
-- General: adjacency implies
distinct vertices
example (V : Type*) (G :
  SimpleGraph V)
  (u v : V) (h : G.Adj u v) : u
    v :=
  G.ne_of_adj h

-- Concrete: path3 adjacency
example : path3.Adj 0 1 := by simp
  [path3]
example : path3.Adj 1 2 := by simp
  [path3]
example : ¬ path3.Adj 0 2 := by
  simp [path3]

-- General: no vertex is adjacent
to itself
example (V : Type*) (G :
  SimpleGraph V)
  (v : V) : ¬ G.Adj v v :=
  G.loopless v
```

① Adj

```

-- General: adjacency implies
distinct vertices
example (V : Type*) (G :
  SimpleGraph V)
  (u v : V) (h : G.Adj u v) : u
    v :=
  G.ne_of_adj h

-- Concrete: path3 adjacency
example : path3.Adj 0 1 := by simp
  [path3]
example : path3.Adj 1 2 := by simp
  [path3]
example : ¬ path3.Adj 0 2 := by
  simp [path3]

-- General: no vertex is adjacent
to itself
example (V : Type*) (G :
  SimpleGraph V)
  (v : V) : ¬ G.Adj v v :=
  G.loopless v

```

① Adj

②

```

-- General: adjacency implies
distinct vertices
example (V : Type*) (G :
  SimpleGraph V)
  (u v : V) (h : G.Adj u v) : u
    v :=
  G.ne_of_adj h

-- Concrete: path3 adjacency
example : path3.Adj 0 1 := by simp
  [path3]
example : path3.Adj 1 2 := by simp
  [path3]
example : ¬ path3.Adj 0 2 := by
  simp [path3]

-- General: no vertex is adjacent
to itself
example (V : Type*) (G :
  SimpleGraph V)
  (v : V) : ¬ G.Adj v v :=
  G.loopless v

```

① Adj

②

③

```
-- General: adjacency implies
distinct vertices
```

```
example (V : Type*) (G :
  SimpleGraph V)
  (u v : V) (h : G.Adj u v) : u
    v :=
  G.ne_of_adj h
```

```
-- Concrete: path3 adjacency
```

```
example : path3.Adj 0 1 := by simp
[path3]
```

```
example : path3.Adj 1 2 := by simp
[path3]
```

```
example : ¬ path3.Adj 0 2 := by
simp [path3]
```

```
-- General: no vertex is adjacent
to itself
```

```
example (V : Type*) (G :
  SimpleGraph V)
  (v : V) : ¬ G.Adj v v :=
  G.loopless v
```

- ① Adj
- ②
- ③
- ④ simp / decide

SimpleGraph

SimpleGraph

```
-- ne_of_adj is a general theorem
#check @SimpleGraph.ne_of_adj
-- {V} (G : SimpleGraph V) {u v :
  V},
--   G.Adj u v → u = v

-- Conjunction of adjacency facts
example :
  path3.Adj 0 1 ∧ path3.Adj 0 2
  :=
  by simp [path3], by simp [path3]

-- Complete graph: u = v → .Adj u v
example (V : Type*) [DecidableEq V]
  (u v : V) (h : u = v) :
  ( : SimpleGraph V).Adj u v :=
  SimpleGraph.top_adj.mpr h
```

SimpleGraph

```
-- ne_of_adj is a general theorem
#check @SimpleGraph.ne_of_adj
-- {V} (G : SimpleGraph V) {u v :
  V},
--   G.Adj u v → u = v

-- Conjunction of adjacency facts
example :
  path3.Adj 0 1  → path3.Adj 0 2
  :=
  by simp [path3], by simp [path3]

-- Complete graph: u = v → .Adj u v
example (V : Type*) [DecidableEq V]
  (u v : V) (h : u = v) :
  ( : SimpleGraph V).Adj u v :=
  SimpleGraph.top_adj.mpr h
```

” ” ” ”

SimpleGraph

```
-- ne_of_adj is a general theorem
#check @SimpleGraph.ne_of_adj
-- {V} (G : SimpleGraph V) {u v :
  V},
--   G.Adj u v → u ~ v

-- Conjunction of adjacency facts
example :
  path3.Adj 0 1 ~ path3.Adj 0 2
  :=
  by simp [path3], by simp [path3]

-- Complete graph: u ~ v ↔ G.Adj u v
example (V : Type*) [DecidableEq V]
  (u v : V) (h : u ~ v) :
  (G : SimpleGraph V).Adj u v :=
  SimpleGraph.top_adj.mpr h
```

• ne_of_adj

" " " "

SimpleGraph

```
-- ne_of_adj is a general theorem
#check @SimpleGraph.ne_of_adj
-- {V} (G : SimpleGraph V) {u v :
  V},
--   G.Adj u v → u    v

-- Conjunction of adjacency facts
example :
  path3.Adj 0 1    ¬ path3.Adj 0 2
  :=
  by simp [path3], by simp [path3]

-- Complete graph: u    v    .Adj u v
example (V : Type*) [DecidableEq V]
  (u v : V) (h : u    v) :
  ( : SimpleGraph V).Adj u v :=
  SimpleGraph.top_adj.mpr h
```

- ne_of_adj

-

” ” ” ”

SimpleGraph

```

-- ne_of_adj is a general theorem
#check @SimpleGraph.ne_of_adj
-- {V} (G : SimpleGraph V) {u v :
  V},
--   G.Adj u v → u ~ v
--
-- Conjunction of adjacency facts
example :
  path3.Adj 0 1 ~ path3.Adj 0 2
  :=
  by simp [path3], by simp [path3]
--
-- Complete graph: u ~ v → G.Adj u v
example (V : Type*) [DecidableEq V]
  (u v : V) (h : u ~ v) :
  (SimpleGraph V).Adj u v :=
  SimpleGraph.top_adj.mpr h

```

- ne_of_adj
-
- $u \quad v \quad .Adj \quad u$
 v

SimpleGraph

```

-- ne_of_adj is a general theorem
#check @SimpleGraph.ne_of_adj
-- {V} (G : SimpleGraph V) {u v :
  V},
--   G.Adj u v → u ~ v

-- Conjunction of adjacency facts
example :
  path3.Adj 0 1 ~ path3.Adj 0 2
  :=
  by simp [path3], by simp [path3]

-- Complete graph: u ~ v → G.Adj u v
example (V : Type*) [DecidableEq V]
  (u v : V) (h : u ~ v) :
  (G : SimpleGraph V).Adj u v :=
  SimpleGraph.top_adj.mpr h

```

- ne_of_adj
-
- u v .Adj u
- v
-
- " " " "

Outline

① G / R / K / C

- ① $G / R / K / C$
- ② AddGroup Group CommRing Field Module Category

- 1 `G / R / K / C`
- 2 `AddGroup Group CommRing Field Module Category`
- 3 `#print`

instance

- ① `G / R / K / C`
- ② `AddGroup Group CommRing Field Module Category`
- ③ `#print`
- ④ `carrier`

- ① `G / R / K / C`
- ② `AddGroup Group CommRing Field Module Category`
- ③ `#print`
- ④ `carrier`
- ⑤ `add/mul/zero/one/smul/Hom/id/comp`

- ① `G / R / K / C`
- ② `AddGroup Group CommRing Field Module Category`
- ③ `#print`
- ④ `carrier`
- ⑤ `add/mul/zero/one/smul/Hom/id/comp`
- ⑥ `#synth`

- ① `G / R / K / C`
- ② `AddGroup Group CommRing Field Module Category`
- ③ `#print`
- ④ `carrier`
- ⑤ `add/mul/zero/one/smul/Hom/id/comp`
- ⑥ `#synth`
- ⑦ `#check`

- ① `G / R / K / C`
- ② `AddGroup Group CommRing Field Module Category`
- ③ `#print`
- ④ `carrier`
- ⑤ `add/mul/zero/one/smul/Hom/id/comp`
- ⑥ `#synth`
- ⑦ `#check`
- ⑧ `Al Lean`